



សាកលវិទ្យាល័យភូមិន្ទភ្នំពេញ
Royal University of Phnom Penh

មហាវិទ្យាល័យវិស្វកម្ម
Faculty of Engineering

ដេប៉ាតឺម៉ង់វិស្វកម្មទូរគមនាគមន៍ និងអេឡិចត្រូនិច
Department of Telecommunications and Electronics Engineering

របាយការណ៍៖
Digital Walkie-Talkie V1 Development

និស្សិត : គួច ម៉េងស្រីន (Kouch Mengsrin)

ថ្ងៃទី.....២៦.....ខែ.....មីនា.....ឆ្នាំ.....២០២៦.....

Abstract

Analog walkie-talkies provide simple and reliable voice communication over long distances. However, advances in communication technology made this method of communication have significant limitations in security and signal protection. Analog modulation techniques such as Frequency Modulation (FM) allow transmitted signals to be easily demodulated by unintended receivers. As a result, Digital Mobile Radio (DMR) systems have become increasingly popular because they use digital modulation schemes that are more resistant to interception. In addition, digital transmission enables the use of encryption techniques, ensuring that voice communication remains private and secure.

This report describes the development of a digital walkie-talkie using a generic ESP32 and an nRF24L01+ with PA/LNA radio transceiver module. As a continuation of previous work, this report focuses solely on the final working implementation, highlighting the refined hardware configuration and optimized software design. The prototype demonstrates the feasibility of implementing digital radio communication using widely available microcontrollers and RF modules. Although reliable communication was achieved, performance is limited by the short range, restricted transmit power, and interference within the 2.4 GHz band. Future implementations intend on addressing these issues by increasing the RF power, and moving the operating frequency down to the VHF/UHF band. An encryption software will also be implemented in the next upcoming work.

Table of Content

1. Introduction	1
2. Development Process	1
2.1 System Overview.....	1
2.2 Hardware Design	3
2.3 Software Design	5
2.4 Testing	11
2.5 Fabrication	12
3. Results & Discussion	15
3.1 Range Test	15
3.2 Current Consumption.....	18
3.3 Limitations and Next Work.....	19
4. Conclusion.....	19

1. Introduction

Wireless voice communication has been widely used in applications such as emergency services, industrial coordination, and personal communication. Traditional walkie-talkies commonly use analog communication methods due to their simplicity and reliability. However, analog communication systems are limited by its security. Transmitted signals can be easily captured and demodulated by unintended receivers operating in the same frequency range. As a result, conversations can be intercepted by nearby devices if the operating frequency is known. This lack of privacy presents a significant limitation, particularly in fields such as military operations and security services, where confidential communication is essential.

Digital Mobile Radios or DMRs has been developed to improve communication security. By using digital modulation techniques such as Frequency Shift Keying (FSK), Gaussian Frequency Shift Keying (GFSK), and Phase Shift Keying (PSK), signal demodulation becomes more complex for unintended receivers. Moreover, with digital data, different encryption algorithms, like the Advanced Encryption Standard (AES), can be utilized to enhance communication security.

This phase of the project will focus on implementing the fundamental principles of digital voice communication using available microcontrollers and radio modules. The current version of the digital walkie-talkie prototype was developed using an ESP32 and an nRF24L01+ with PA/LNA transceiver module. The overall objective of the project is to demonstrate a digital voice communication system that is reliable, simple, and more secure than traditional analog walkie-talkies.

For the current development phase, the objectives are:

- Understand how digital voice communication works
- Basic transmission of digital voice data over RF
- Construct a PCB prototype for the digital walkie-talkie design

2. Development Process

2.1 System Overview

Walkie-talkies are half-duplex communication devices, meaning both users can transmit and receive signals, but not simultaneously. Communication is controlled through a push-to-talk (PTT) mechanism, which switches the device between transmit and receive modes. For an analog walkie-talkie, speech is captured via a microphone and converted to electrical signals. This signal is then sent to an analog modulation circuit to modulate a carrier frequency for signal transmission. The signal is transmitted using an antenna. On the receiving side, the transmitted signal is captured via the antenna, demodulated then amplified and sent to a speaker for sound output.

The figure below sketches the essential parts of a walkie-talkie:

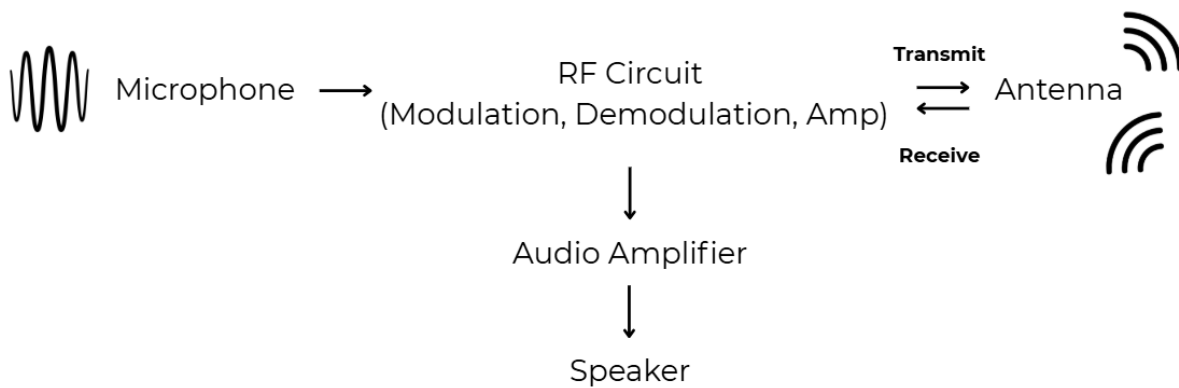


Figure 1. Operation of a Traditional Walkie-Talkie

The operation is similar for a digital walkie-talkie but involves additional signal processing stages. Because digital walkie-talkies use digital modulation schemes, after voice signals are captured by the microphone, it needs to be converted to digital signals through sampling, quantization and encoding. The resulting digital data is then modulated using a digital modulation scheme and transmitted. At the receiver, the signal is demodulated, decoded, and converted back to an analog signal for audio output through a speaker.

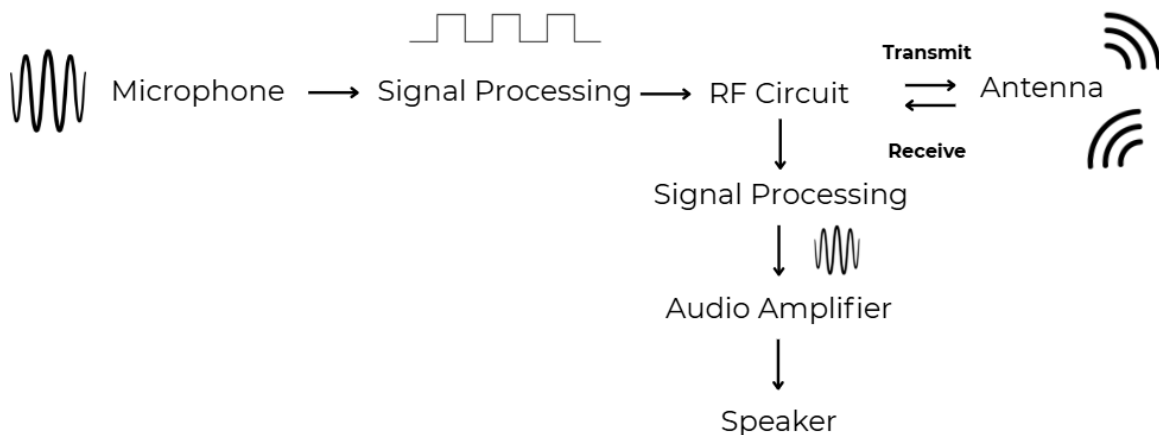


Figure 2. Operation of a Digital Walkie-Talkie

Observing the components of a digital walkie-talkie, there are a few key points to consider when developing the digital walkie-talkie:

- **Signal Processing:** What should be used to sample the voice signals?
- **RF circuit:** This part of the circuit should be capable performing digital modulation and demodulation, and creating a strong enough RF signal for transmission.
- **Audio Amplifier:** Capable of amplifying audio without noise or distortion.

All of these points shall be taken into consideration when choosing the right hardware components for the prototype.

2.2 Hardware Design

The components used in this project were chosen to simplify the development process of this prototype, which is why all of the components used are ready-made modules. Reasons for choosing each component are presented as follows.

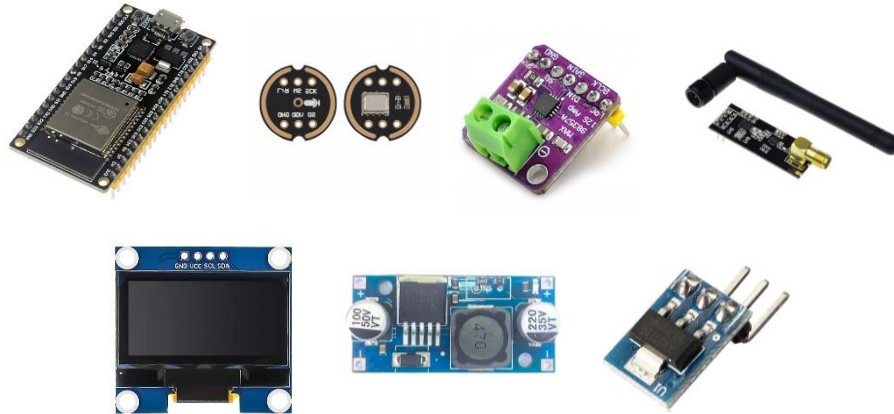


Figure 3. List of Components Used in This Project

- **ESP32 38-Pin**

The microcontroller will act as the “brain” of the digital walkie-talkie. It will be the one performing signal processing and controlling the overall hardware of the circuit.

This microcontroller was selected as it offers high processing speed, enabling efficient real-time signal processing. It is easily programmable using the Arduino IDE, with many built-in libraries, such as the **RF24.h** library for the nRF module, that simplify development and integration. Additionally, the platform is highly scalable, allowing for future expansion and the incorporation of advanced features.

- **INMP441 I²S Microphone**

A digital microphone used to capture voice signals and directly output them in digital form. Unlike traditional analog microphones, it integrates an internal analog-to-digital converter (ADC), eliminating the need for external circuitry to perform sampling, quantization, and encoding. This simplifies the overall system design and reduces processing on the microcontroller. The microphone utilizes the **Inter-IC Sound (I²S)** protocol, a digital communication interface specifically designed for transmitting digital audio data between integrated circuits in a synchronized format. This ensures that the audio output is high-quality and are not be subjected to noise or distortion.

- **MAX98357A I²S Amplifier**

A digital audio amplifier that receives audio data via the I2S interface and converts it into analog signals to drive a speaker. It integrates both a digital-to-analog converter (DAC) and a Class D amplification stage, allowing direct speaker output without requiring additional external components. This integration simplifies the overall circuit design of this prototype.

- **nRF24L01+ with PA/LNA**

An RF transceiver module that operates at the frequency of 2.4GHz with very low power consumption. The additional PA/LNA (Power amplifier/Low-noise amplifier) included in the module improves the transmission power and receiving sensitivity.

Specifications from the nRF24L01+ chip's datasheet:

- Worldwide 2.4GHz ISM band operation
- **126 RF channels** ($f = 2400 + RF_CH[MHz]$). So, Channel 1 would be 2401MHz and Channel 10 would be 2410MHz)
- Uses **GFSK Modulation**
- **250kbps, 1Mbps and 2Mbps** air data rate (How many bits can be transmitted and processed per second)
- Maximum output power of **0dBm** (But with the PA/LNA modules, it claims to have an output power of +20dBm)
- Transmission current consumption: **11.3mA at 0dBm**
- Receiving current consumption: **13.5mA at 0dBm**
- Receiving sensitivity: **-94dBm at 250kbps**
- **1 to 32 bytes** dynamic payload length (can send a maximum of 32 bytes of data at a time)
- Supply voltage: **3.3V (typ.)**



nRF24L01+ Product Specification

1.1 Features

Features of the nRF24L01+ include:

- Radio
 - Worldwide 2.4GHz ISM band operation
 - 126 RF channels
 - Common RX and TX interface
 - GFSK modulation
 - 250kbps, 1 and 2Mbps air data rate
 - 1MHz non-overlapping channel spacing at 1Mbps
 - 2MHz non-overlapping channel spacing at 2Mbps
- Transmitter
 - Programmable output power: 0, -6, -12 or -18dBm
 - 11.3mA at 0dBm output power
- Receiver
 - Fast AGC for improved dynamic range
 - Integrated channel filters
 - 13.5mA at 2Mbps
 - -82dBm sensitivity at 2Mbps
 - -85dBm sensitivity at 1Mbps
 - -94dBm sensitivity at 250kbps
- RF Synthesizer
 - Fully integrated synthesizer
 - No external loop filter, VCO varactor diode or resonator
 - Accepts low cost ±60ppm 16MHz crystal
- Enhanced ShockBurst™
 - 1 to 32 bytes dynamic payload length
 - Automatic packet handling
 - Auto packet transaction handling
 - 6 data pipe MultiCaster™ for 1:8 star networks
- Power Management
 - Integrated voltage regulator
 - 1.9 to 3.6V supply range
 - Idle modes with fast start-up times for advanced power management
 - 26µA Standby-I mode, 900nA power down mode
 - Max 1.5ms start-up from power down mode
 - Max 130µs start-up from standby-I mode
- Host Interface
 - 4-pin hardware SPI
 - Max 10Mbps
 - 3 separate 32 bytes TX and RX FIFOs
 - 5V tolerant inputs
- Compact 20-pin 4x4mm QFN package

SPI RF-SETUP (RF_PWR)	RF output power	DC current consumption
11	0dBm	11.3mA
10	-6dBm	9.0mA
01	-12dBm	7.5mA
00	-18dBm	7.0mA

Datarate	Symbol	Parameter (condition)	Notes	Min.	Typ.	Max.	Units
	RX _{max}	Maximum received signal at <0.1% BER			0		dBm
2Mbps	RX _{SENS}	Sensitivity (0.1%BER) @2Mbps			-82		dBm
1Mbps	RX _{SENS}	Sensitivity (0.1%BER) @1Mbps			-85		dBm
250kbps	RX _{SENS}	Sensitivity (0.1%BER) @250kbps			-94		dBm

Symbol	Parameter (condition)	Notes	Min.	Typ.	Max.	Units
f _{OP}	Operating frequency	a	2400		2525	MHz
PLL _{res}	PLL Programming resolution			1		MHz
f _{TAL}	Crystal frequency			16		MHz
Δf ₂₅₀	Frequency deviation @ 250kbps			±160		kHz
Δf _{1M}	Frequency deviation @ 1Mbps			±160		kHz
Δf _{2M}	Frequency deviation @ 2Mbps			±320		kHz
R _{GFSK}	Air Data rate	b	250		2000	kbps
F _{CHANNEL 1M}	Non-overlapping channel spacing @ 250kbps/1Mbps	c		1		MHz
F _{CHANNEL 2M}	Non-overlapping channel spacing @ 2Mbps	c		2		MHz

Figure 4. nRF24L01+ Specifications

This chip has 20 pins but the module that uses this chip only has 8 pins to connect: 2 pins for VCC and GND, the other 6 are for SPI communication.

- VCC and GND
- CE: Chip enable activates RX or TX mode

- CSN: SPI Chip Select
- SCK: SPI Clock
- MOSI: SPI Slave Data Input
- MISO: SPI Slave Data Output
- IRQ: Interrupt pin (Not used in this project's case)

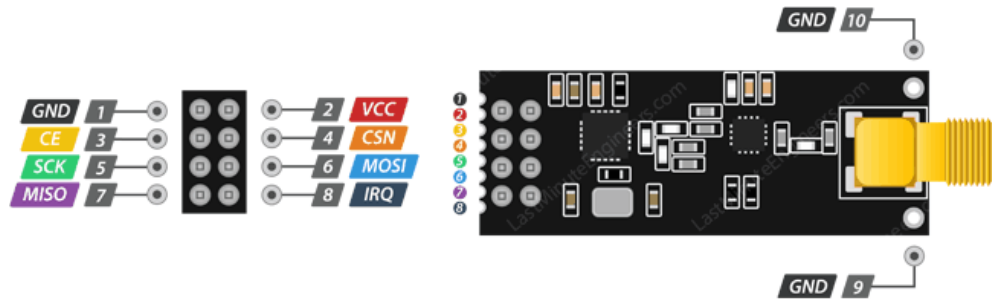


Figure 5. nRF24L01+ with PA/LNA module Pinout

Due to its ease of customization and programmability with the microcontroller, this module was selected as the “RF circuit” of the design.

• Other Components

- OLED screen display to provide visual feedback
- A potentiometer with a switch (5-pin pot) to control the volume of the audio output and to switch the walkie-talkie on and off
- A button for the PTT feature
- Power supply circuit: battery, 5V buck converter for powering the ESP32 and other 5V components, and a 3.3V regulator for supplying the nRF module

(Each individual component was tested with the microcontroller to ensure each component is fully functional.)

The circuit was built by combining these components together, connecting each module to the ESP32 GPIO pins while ensuring that all ground connections are connected so that everything is referenced to the same ground. Pin connections are chosen carefully to avoid pins that are reserved or have special boot functions on the ESP32.

2.3 Software Design

There were a few key points that was considered to guide the development of the code or program of the digital walkie-talkie:

- Radio communication control: how to handle real-time data transmission, efficient packet handling and latency management (Walkie-talkies require low latency for real-time communication)? How to switch between TX and RX mode using PTT mechanism?
- Interfacing with I²S: how does the I²S microphone and audio amplifier handle audio?
- Sampling rate and bit-depth consideration for audio (Whether the nRF could handle sending high-quality audio)

- Buffering: how to ensure continuous audio streaming without voice data loss?
- Error handling and packet loss: How to handle dropped packets or corrupted packets?

Walkie-talkie code implementation

(All of the tasks and functions in this code are closely related and can't truly be separated from each other. Therefore, the entire code will be explained at once.)

There are 4 main parts of this code implementation:

- Radio communication
- Audio implementation
- Channel Selection
- Controls and visual feedback

Necessary library headers are defined: Libraries for the ESP32, OLED screen display, SPI communications, nRF24L01+ module, I²S peripherals and FreeRTOS (Real-time operating system) that allows the ESP to execute multiple tasks concurrently in a controlled and efficient way. FreeRTOS is the main component that allows the ESP32 to handle multiple tasks at once.

```
#include <Arduino.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <SPI.h>
#include <RF24.h>
#include <driver/i2s.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/adc.h"
```

Pin definitions: the numbers are GPIO number

```
// OLED
#define SDA_PIN 21
#define SCK_PIN 23

// Hardware Controls
#define BUTTON 15
#define LED_PIN 5
#define POT_CHANNEL ADC1_CHANNEL_6

// INMP441 I2S Mic
#define I2S_WS 25
#define I2S_SD 33
#define I2S_SCK 32
```

```
// MAX98357A Amp
#define MAX98357A_LRCLK 18
#define MAX98357A_BCLK 19
#define MAX98357A_DIN 22

// nRF24L01 SPI
#define NRF_SCK 14
#define NRF_MISO 12
#define NRF_MOSI 13

#define CE_PIN 2
#define CSN_PIN 4
```

Global variables and states:

- Display control variables and system states (DisplayState)
- Timing variables for animations, delays, and state transitions
- Radio communication buffers (TX double buffer and RX ring buffer)
- Audio-related parameters such as sample rate, gain, and thresholds
- ADPCM encoder state for maintaining continuity between audio frames

```
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);
Preferences preferences;
uint8_t savedChannel = 60;

enum DisplayState { STATE_IDLE, STATE_TRANSMIT, STATE_RECEIVE, STATE_VOLUME,
STATE_STARTUP, STATE_SCREEN_OFF, STATE_CHANNEL };
volatile DisplayState displayState = STATE_STARTUP;

String welcomeText = "Welcome to TEE! ";
int16_t scrollX = SCREEN_WIDTH;
const uint8_t scrollSpeed = 5;
uint8_t dotCount = 0;

// Timers
unsigned long lastVolumeChange = 0;
unsigned long lastReceiveTime = 0;
unsigned long lastAnimUpdate = 0;
unsigned long startupTime = 0;
unsigned long lastChannelActivity = 0;
const unsigned long VOLUME_SHOW_DURATION = 2000;
const unsigned long SCREEN_OFF_DELAY = 1000;
const unsigned long STARTUP_SHOW_DURATION = 1500;
const unsigned long RECEIVE_TIMEOUT = 500;
const unsigned long ANIM_INTERVAL = 300;
```

```

const unsigned long CHANNEL_SHOW_DURATION = 1500;
const unsigned long BUTTON_HOLD_DURATION = 700;
const unsigned long RAPID_CHANNEL_SWITCH_TIME = 200;

// Radio & Buffers
uint8_t currentChannel = 80;
volatile bool channelUpdatePending = false;
const uint8_t MAX_CHANNEL_VALUE = 125;
const uint8_t MIN_CHANNEL_VALUE = 80;

#define SAMPLE_RATE 16000
#define PACKET_SIZE 32
#define ADPCM_HEADER_SIZE 3
#define ADPCM_DATA_BYTES (PACKET_SIZE - ADPCM_HEADER_SIZE)
#define SAMPLES_PER_FRAME (ADPCM_DATA_BYTES * 2) // 58 samples per packet
const uint8_t SHARED_ADDRESS[6] = "Srun";

SPIClass spiNRF(HSPI);
RF24 radio(CE_PIN, CSN_PIN);

uint8_t txBuffer[2][PACKET_SIZE];
volatile bool bufferReady[2] = {false, false};
volatile uint8_t fillBuffer = 0;
volatile uint8_t txBufferIndex = 0;
volatile int txFails = 0;
volatile bool isTransmitting = false;

// Receive-side jitter ring buffer
#define RX_RING_SIZE 18
#define RX_PREFILL 2 //How many packets to fill before sending to amp
uint8_t rxRing[RX_RING_SIZE][PACKET_SIZE];
volatile uint8_t rxHead = 0;
volatile uint8_t rxTail = 0;
volatile uint8_t rxCount = 0;
volatile bool rxPlaying = false;
portMUX_TYPE rxCountMux = portMUX_INITIALIZER_UNLOCKED;

// ADPCM encoder state (persistent across frames during TX)
typedef struct { int16_t predicted; uint8_t index; } ADPCMState;
ADPCMState txAdpcmState = {0, 0};

// Audio Gain Controls
int lastPotValue = -1;
const int VOLUME_HYSTERESIS = 400;

```

```

// Display Float vs Fast Integer Math
volatile float GAIN_FOR_UI = 1.0f;           // Kept for drawing the OLED
volume bar
volatile int32_t GAIN = 256;                // OPTIMIZED: 1.0 = 256 for fast audio math
const float GAIN_MIN = 0.1f;
const float GAIN_MAX = 3.5f;

// Limiter thresholds
const int32_t LIMIT_PRE = 10000;
const int32_t LIMIT_POST = 28000;

// Raise this value if still getting feedback; lower it if soft speech is cut
off.
const int32_t NOISE_GATE_THRESHOLD = 300; // range: 0 (disabled) - ~32767 (full-
scale); practical: 50-500
const int32_t FIXED_MIC_GAIN = 179; // 0.7x gain (256 = 1.0x)

```

For the nRF module, 250kbps data rate will give the best maximum range. Therefore, the sample rate was chosen as 16kHz to compromise for that data rate setting.

At 16kHz, if we sample using raw PCM (Pulse code modulation) at 16-bit per sample, the data rate would come out to be 256kbps. That is way above the 250kbps of the nRF. Thus, a compression algorithm was used to compress the audio down to 4 bits while maintaining roughly the same audio quality.

The algorithm used was **ADPCM** (Adaptive Differential Pulse Code Modulation). Instead of sending full audio samples, it only sends the difference between consecutive samples.

By reducing the data sent from 16 bits to 4 bits, the data rate drops to ~64kbps, well within the rate of 250kbps.

- Radio communication

Radio communication is handled inside the radioTask(), which continuously runs as a FreeRTOS task. The system operates in two modes: transmit and receive, controlled by the isTransmitting flag.

In transmit mode, the task checks if a buffer (bufferReady) contains encoded audio data. If available, it sends the packet using radio.write(). A double-buffering system (txBuffer[2]) is used so that one buffer can be filled while the other is being transmitted. If transmission fails multiple times, the radio module is reset using powerDown() and powerUp().

In receive mode, the task continuously checks radio.available() for incoming packets. Received data is stored in a circular (ring) buffer (rxRing) using rxHead, rxTail, and rxCount. This buffer helps absorb timing variations between incoming packets. The task also handles dynamic channel updates by reconfiguring the radio when channelUpdatePending is set.

- Audio Implementation

Audio input and output are handled by ESP32's I2S driver. The program uses a single I2S port, dynamically switching between microphone input when transmitting (`switchToMicPins()`), and speaker output when receiving (`switchToSpeakerPins()`).

The `audioCaptureTask()` captures raw audio as 24-bit from the INMP441 microphone but then scaled down to 16-bit. The samples are processed using functions like:

- `softLimiter()` prevents signal clipping by compressing large amplitudes
- `hpFilter()` removes microphone noise floor from the audio signal
- A noise gate to suppress low-level signals
- The captured audio is attenuated by 0.7x because the microphone is too sensitive. Reducing the amplitude of the captured signal ensures that most of the audio does not clip and it reduces feedback between the two walkie-talkies when placed close to each other

The processed samples are then compressed using IMA ADPCM (`adpcmEncode`) to reduce data size before transmission.

- ADPCM lookup tables define step sizes and index adjustments
- `adpcmEncode()` compresses PCM audio into 4-bit ADPCM format
- `adpcmDecode()` reconstructs PCM audio from compressed data

The encoded data, along with a small header (predictor and index), is stored in the transmit buffer.

On the receiving side, the `playbackTask()` retrieves packets from the ring buffer, extracts the ADPCM header, and reconstructs the audio using `adpcmDecode()`. The decoded samples are scaled by the current volume, passed through a limiter, and written to the I2S interface using `i2s_write()` for playback through the amplifier.

- Channel Selection

Channel selection is implemented in the `buttonTask()`. Two buttons (`BTN_LEFT` and `BTN_RIGHT`) are used to decrease or increase the channel value within a defined range.

A short press changes the channel by one step, while holding the button enables rapid channel switching using a timed repeat mechanism. Debouncing is handled in software using timing checks.

When a channel is changed, the flag *channelUpdatePending* is set. The actual update is performed in the `radioTask()` to ensure safe reconfiguration of the radio module. The selected channel is saved using the Preferences library, allowing it to persist after a restart.

- Controls and visual feedback

User inputs and system feedback are managed across several tasks.

The `buttonTask()` handles the push-to-talk (PTT) button. When pressed, the system switches to transmit mode by stopping radio listening, flushing buffers, and switching I2S to microphone

input. When released, it returns to receive mode, resets buffers, and switches I2S back to speaker output.

The `volumeReadTask()` reads the potentiometer value using the ADC. Changes in the potentiometer adjust the system gain (GAIN), which is used during audio playback. A hysteresis mechanism is implemented to prevent small fluctuations from triggering unnecessary updates.

The `OLEDTask()` manages all display output. It updates the screen based on the current `displayState`, showing different information such as transmitting, receiving, volume level, channel number, and idle animations. Timing logic is used to control transitions between states.

An LED indicator is also controlled in the `playbackTask()`, turning on when audio is actively being played and off during silence or buffer underruns.

After everything is defined, a few settings and setups are performed in the void `setup()` section.

- Configures GPIO pins and serial communication
- Initializes the OLED display and loads saved channel settings
- Sets up the nRF24L01 radio parameters (power level, data rate, channel).
 - Power level: MAX (0dBm)
 - Data rate: 250kbps
 - Channel: set to 108 (operating at 2508MHz)
- Initializes the I2S driver and ADC for audio and volume control

2.4 Testing

The circuit was first tested on a breadboard:

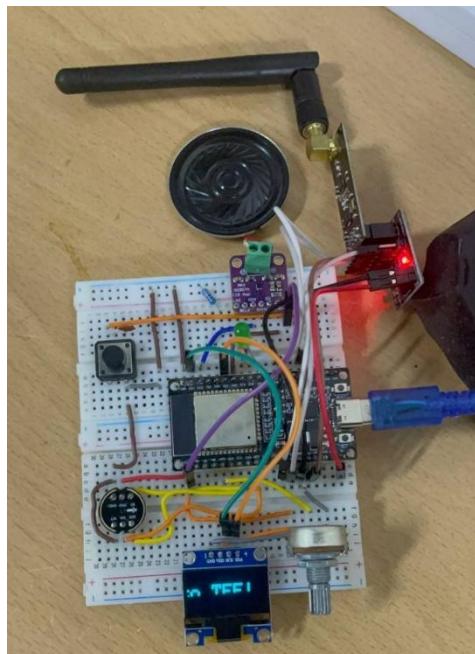


Figure 6. Circuit connection on breadboard

The walkie-talkies were placed on opposite ends of a room to test voice transmission and reception.

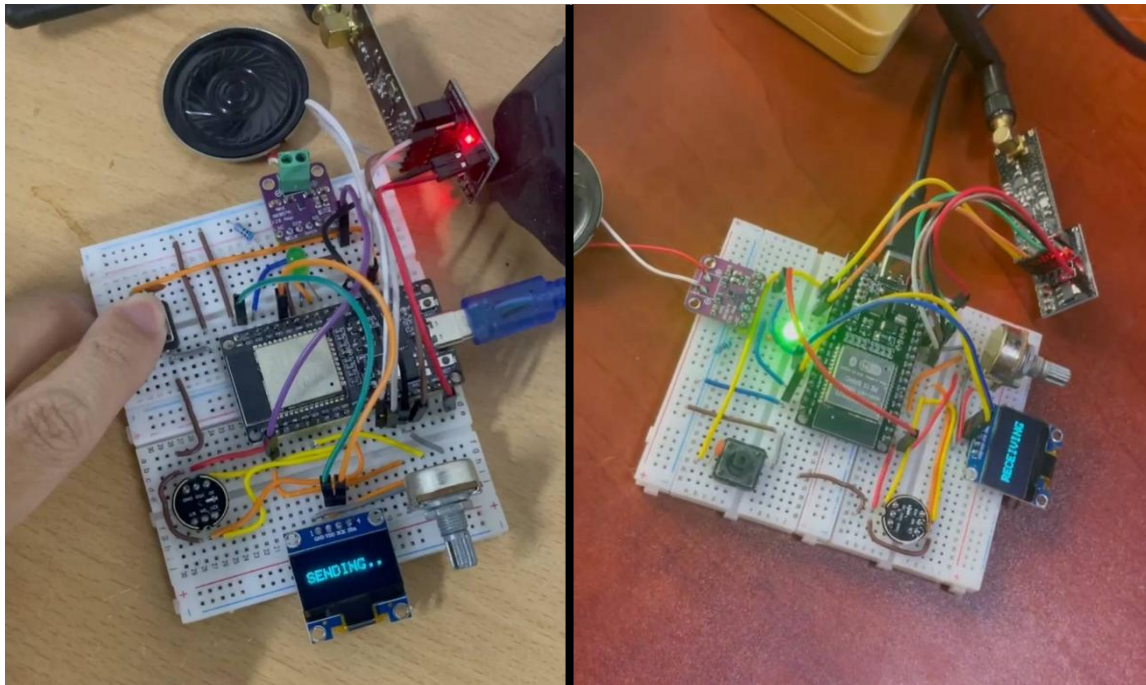


Figure 7. Testing on Breadboard

The test was successful!

2.5 Fabrication

The schematic and PCB layout of the circuit was drawn to prepare the board for manufacturing:

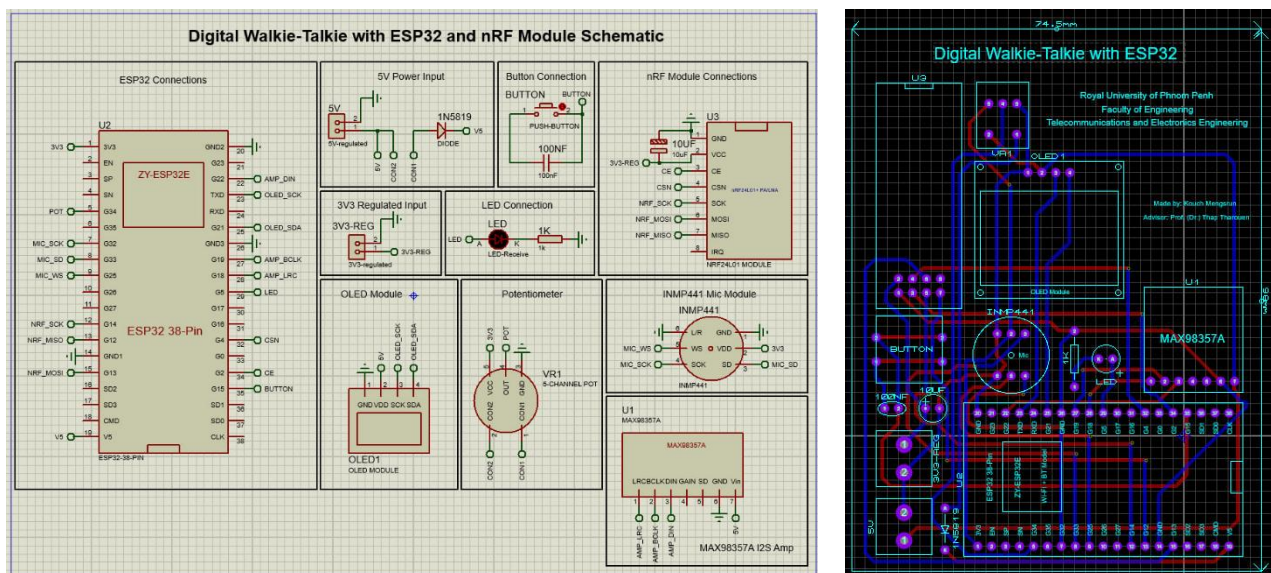


Figure 8. Schematic and PCB layout of the Walkie-talkie



Figure 9. First PCB Fabrication

However, for the first PCB fabrication, wire connections were not properly crosschecked and a pin was wired incorrectly. This meant that a PCB trace had to be manually cut which impacted the board's aesthetic. Moreover, during the PCB layout design, the power supply part of the circuit was not taken into consideration which made holding and testing with the board very difficult.

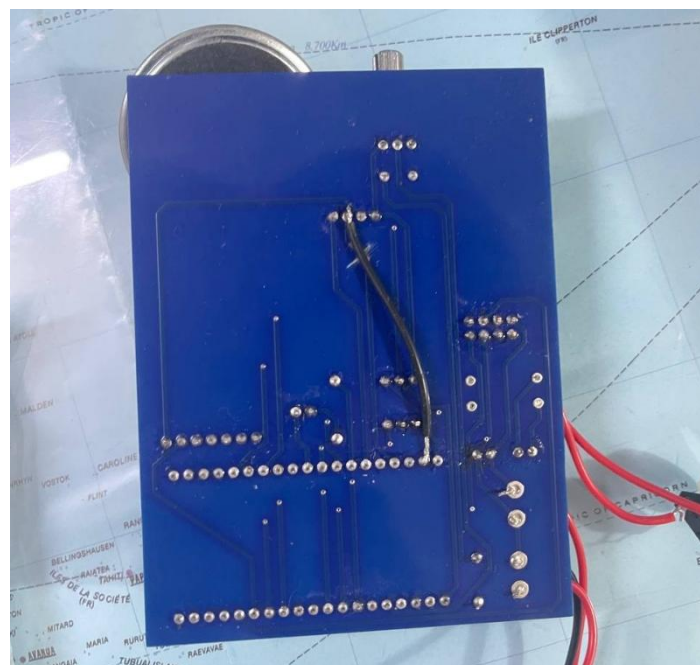


Figure 10. Connection Mistake on the first PCB

To fix these mistakes, the circuit schematic and PCB layout were redrawn. Additionally, 4 buttons were added to the design which enables us to add a channel selection or settings feature later on in future implementations.

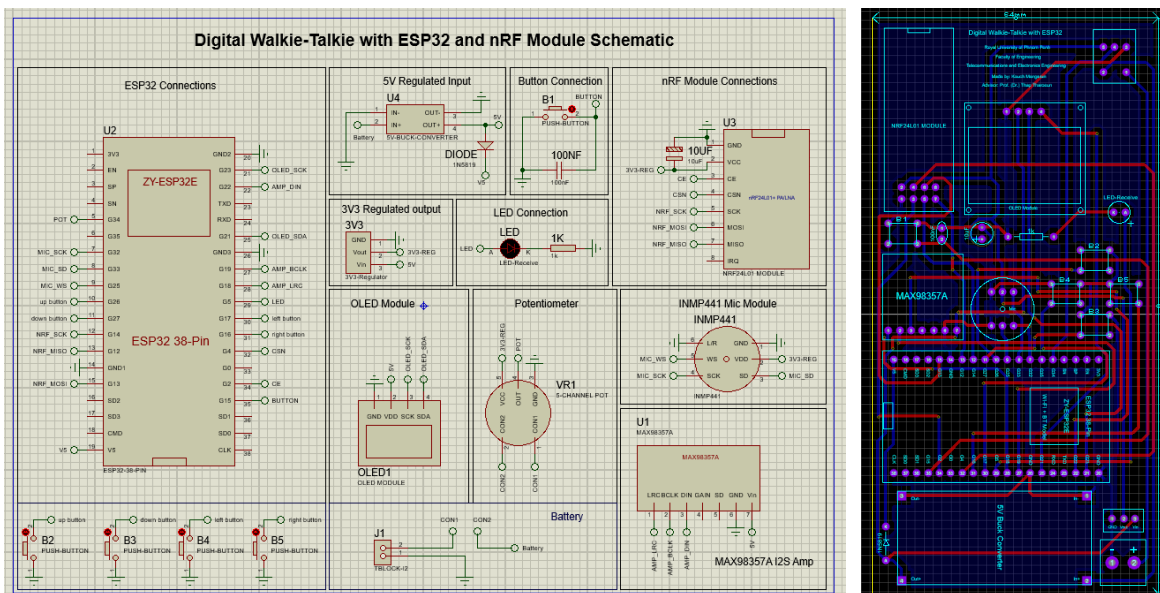


Figure 11. Redrawn Schematic and PCB layout



Figure 12. Second PCB Fabrication

Final PCB look:

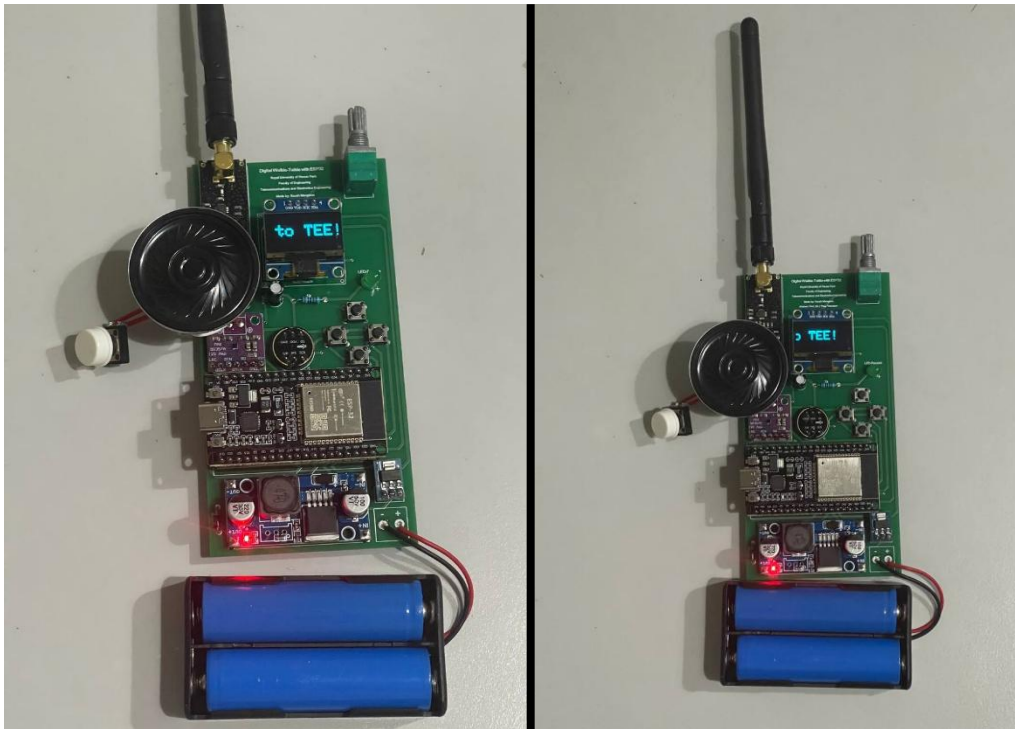


Figure 13. Final PCB Design for Walkie-talkie V1

3. Results & Discussion

3.1 Range Test

The walkie-talkies were tested within our university premises using the PCB.

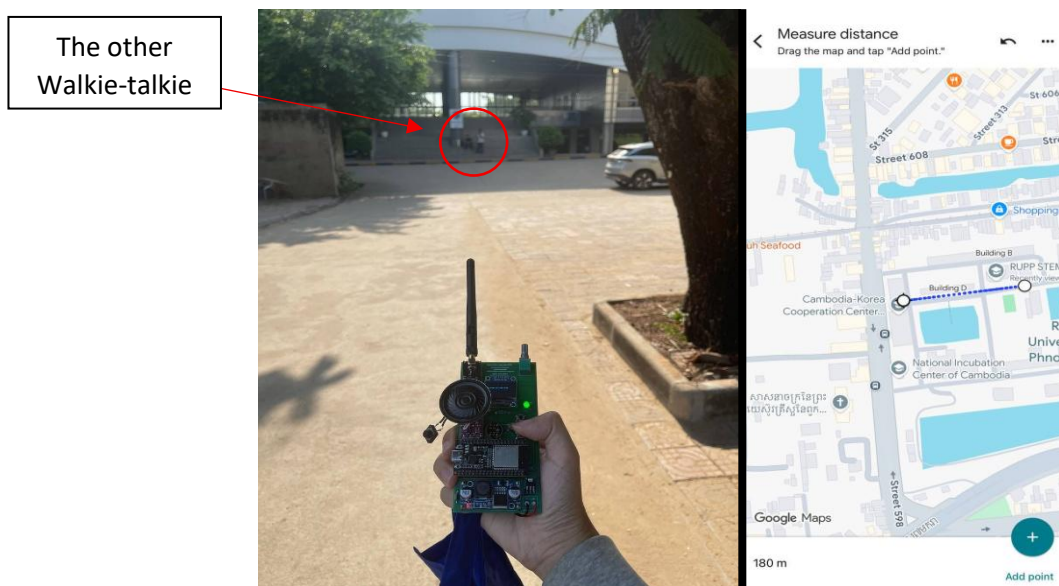
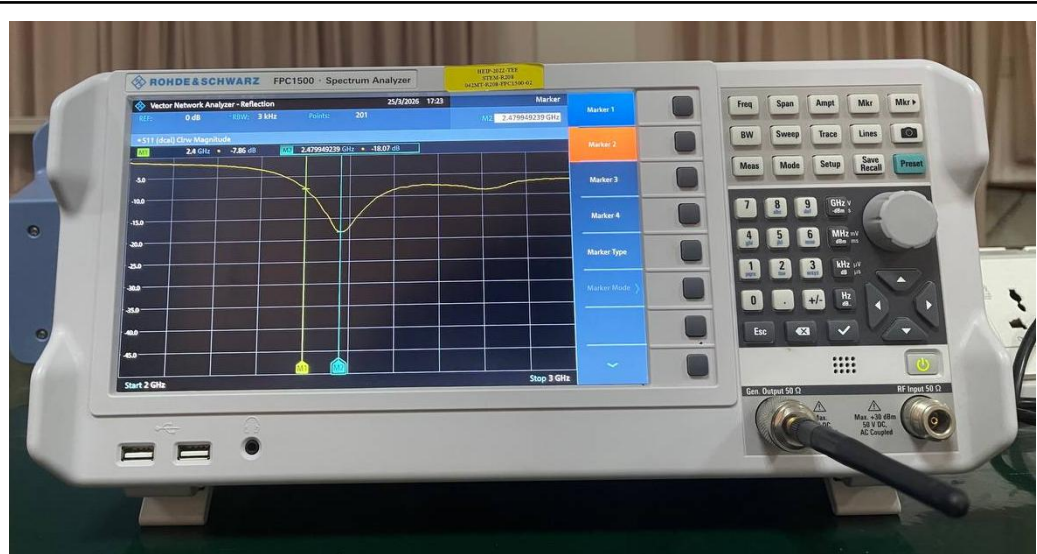


Figure 14. First Range Test

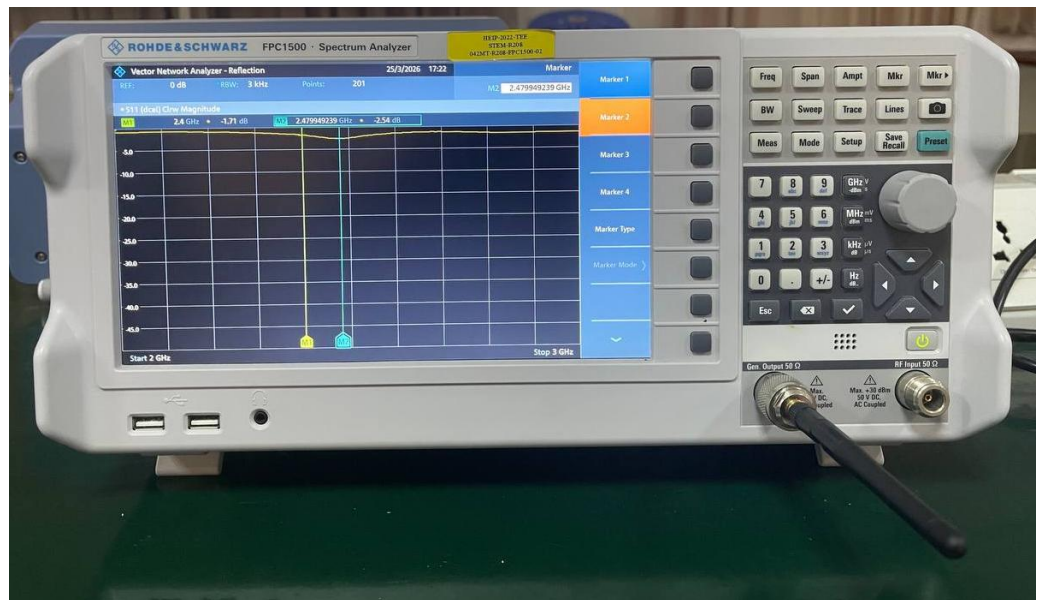
The walkie-talkies were tested from in front of CKCC to in front of STEM building's walkway. Through testing and using google maps "measure distance" feature, the walkie-talkies achieved a measured usable range of **~180 meters line-of-sight** before its performance start degrading.

A few experiments were performed to examine why the range was limited.

The antenna that was provided with the nRF24L01 were tested with a VNA (Vector Network Analyzer) to see its S11 parameter.



(a)



(b)

Figure 15. (a) Normal Antenna S11 Performance, (b) Poor Antenna S11 Performance

Upon observing the S11, it is clear that one of the antennas has poor performance, with a reflection coefficient of -2.54dB at its resonant frequency. This means, during transmission, half of the power is reflected back to the module.

A new antenna was replaced and tested on the VNA.

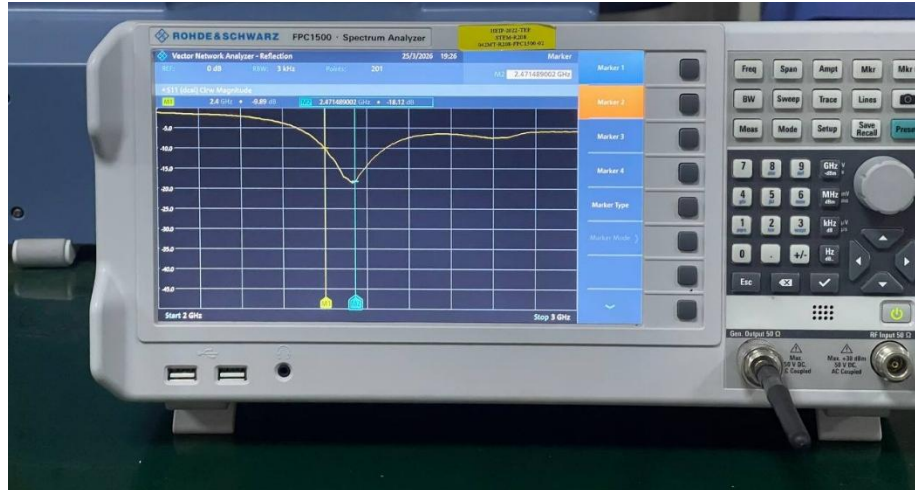


Figure 16. New antenna S11 parameter

Both of the antennas have a resonant frequency of ~ 2.48 GHz, with S11 being below -15dB. Then, in theory, both of the walkie-talkie should work best at **Channel 80** ($2400 + 80$ [MHz]). Initially, the channel number was chosen arbitrarily (Channel 108). With this result, the walkie-talkies were tested again on Channel 80 to confirm if the hypothesis was correct.

Tested from in front of CKCC to Chuon Nath Garden in RUPP:

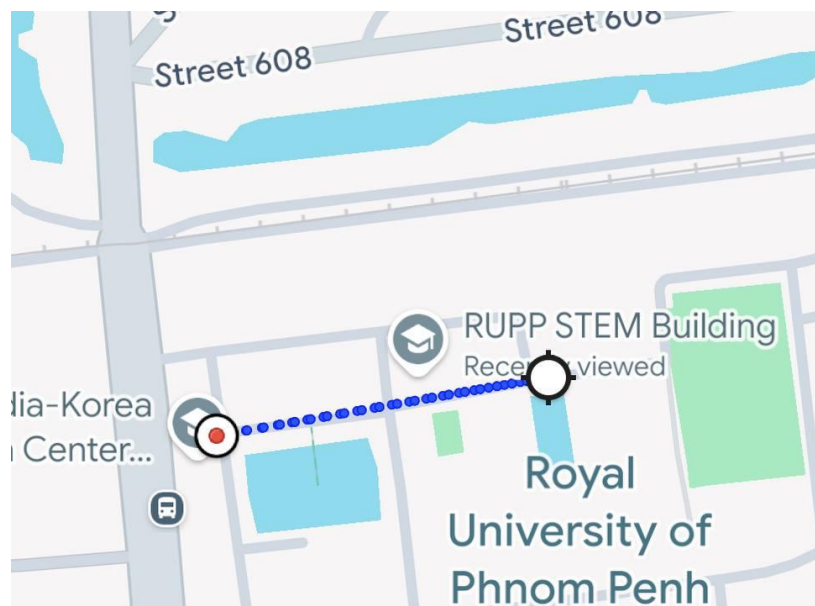


Figure 17. Second Range Test Location

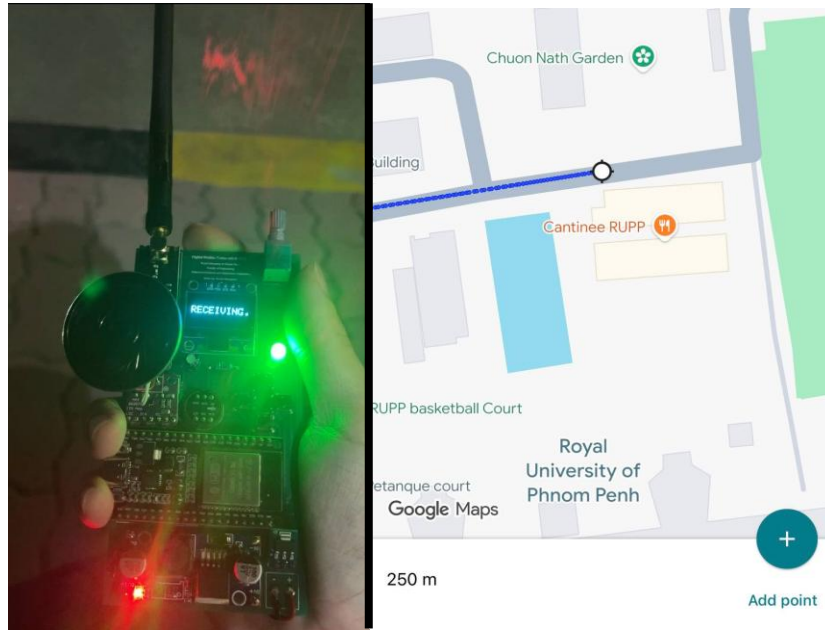


Figure 18. Second Test Achieved Range

On the second test, a range of **250 meters line-of-sight** was achieved, an improvement from the previous test. However, the range is still quite limited.

3.2 Current Consumption

The total current consumption was tested by supplying the walkie-talkie using a power supply and observing the current meter on the power supply display. The entire circuit consumed:

- 87.3mA when idle
- 95mA when transmitting and receiving

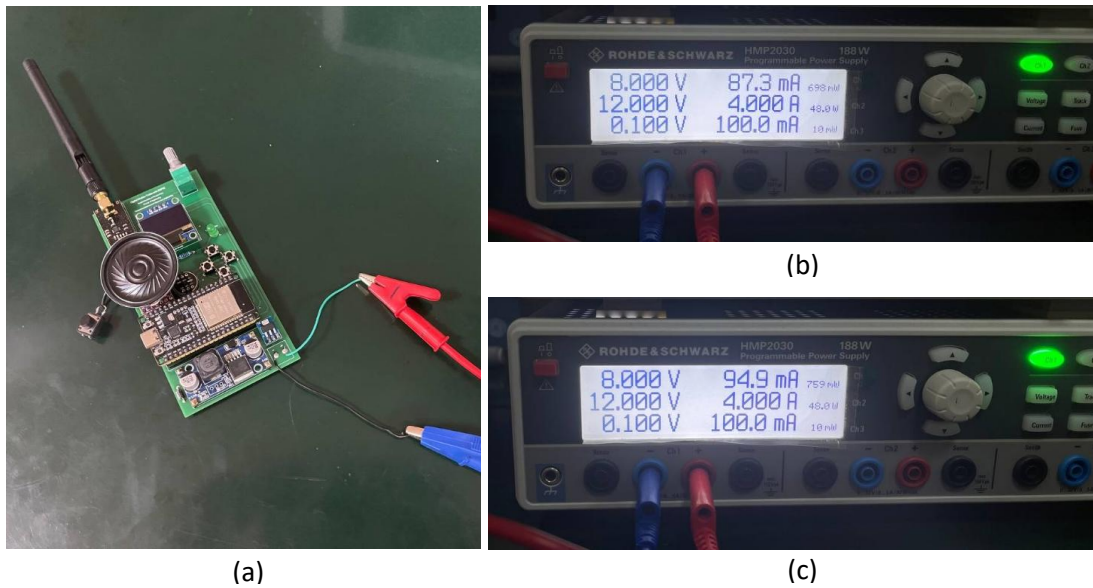


Figure 19. (a) Testing Setup, (b) Current consumption when idle, (c) Current Consumption when Transmitting

3.3 Limitations and Next Work

Limitations:

- 2.4GHz ISM band is heavily congested, with technologies like Bluetooth, WiFi, Mobile service etc. operating in the same band. Therefore, it is susceptible to interference.
- Low transmit power. If we were to increase the transmit power, it would interfere with all the other devices in the same band.
- More path loss, distance is heavily cut short.
- Obstacles will degrade the signal strength heavily

Next step:

- Find a new RF module that offers similar features but operates in the sub-GHz range or in the lower part of UHF band.
- Add AES encryption to make communication even more secure
- Design a software for programming the encryption to our digital walkie-talkie

4. Conclusion

A digital walkie-talkie prototype was successfully developed using an ESP32, nRF24L01 radio module, and I2S audio interface. The system employs GFSK modulation, and high-quality audio transmission was achieved over the nRF link using ADPCM audio compression. The walkie-talkie features volume control, visual feedback via an OLED display, and channel selection within the 2.4 GHz to 2.525 GHz range. In real-world tests, the device demonstrated a usable line-of-sight range of approximately 250 meters.

Despite successful voice transmission, the range remains limited due to the low output power of the nRF24L01 and congestion in the 2.4 GHz ISM band. Future development will focus on integrating RF modules that operate in the lower UHF band with higher output power to extend range. Additionally, AES encryption will be implemented to enhance the security of communications.

GitHub Repository for this version of the project:

<https://github.com/mengsrn-commits/Digital-Walkie-talkie-V1>